Лабораторная работа №4: Динамическое программирование в игре «Монетный ряд»

Слёлин А.В. ПМИ-11МО

27 апреля 2026 г.

Постановка задачи

Необходимо разработать интерактивную консольную игру «Монетный ряд», в которой игрок и компьютер по очереди берут монеты с одного из двух концов ряда. Цель игры — набрать максимальную сумму. Компьютер должен использовать алгоритм динамического программирования для выбора оптимальной стратегии на каждом ходе. Тематика игры неразрывно связана с методом динамического программирования: компьютер вычисляет оптимальный ход, максимизирующий свой выигрыш в предположении, что соперник также играет оптимально.

Входные данные: начальный ряд монет (массив положительных целых чисел). Выходные данные: интерактивный игровой процесс, завершающийся подведением итогового счёта и определением победителя.

Теоретическая часть

Постановка игровой задачи

Имеется ряд из N монет с номиналами $c_0, c_1, \ldots, c_{N-1}$. Игроки ходят по очереди. На каждом ходе игрок выбирает и забирает монету либо с левого конца (индекс L), либо с правого (индекс R), после чего границы ряда сужаются. Игра заканчивается, когда все монеты разобраны. Каждый игрок стремится максимизировать сумму своих монет. Противник считается рациональным и также действует оптимально.

Решение методом динамического программирования

Задача относится к классу антагонистических игр с полной информацией и нулевой суммой (выигрыш одного равен проигрышу другого). Оптимальная стратегия может быть найдена с помощью динамического программирования.

Пусть $dp[i][j]$ — максимальный гарантированный выигрыш игрока, который делает ход первым на подмассиве монет $c_i \ldots c_j$. Тогда рекуррентное соотношение:

$$dp[i][j] = \max \begin{cases} c_i + \min\big(dp[i+2][j],\ dp[i+1][j-1]\big), \\ c_j + \min\big(dp[i+1][j-1],\ dp[i][j-2]\big) \end{cases}$$

Обоснование: если игрок берёт левую монету c_i, то противник на оставшемся подмассиве $i+1 \ldots j$ будет действовать оптимально и оставит нам минимально возможный выигрыш из двух вариантов (в зависимости от его выбора). Аналогично для правой монеты.

Базовый случай: $dp[i][i] = c_i$.

После заполнения таблицы dp для каждого подмассива сохраняется оптимальный первый ход (левая или правая монета). Компьютер, когда наступает

его очередь, просто смотрит в таблицу для текущих границ L и R и делает предписанный ход.

Сложность

Заполнение таблицы dp размером $N \times N$ требует $O(N^2)$ времени и памяти. После этого ответ на любой запрос оптимального хода даётся за $O(1)$. Для $N \leqslant 15$ (как в реализации) это мгновенно.

Алгоритм (реализация)

Программа состоит из нескольких классов:

- `CoinRowGame` — основная логика игры, управление ходами и счётом.
- `DpSolver` (внутренний класс) — реализует динамическое программирование, строит таблицы dp и $choice$.
- `GameController` — консольный интерфейс, обработка ввода пользователя, отображение состояния.
- Вспомогательные классы: `AnsiColors` (ANSI-коды), `Choice` (перечисление LEFT/RIGHT), `Turn` (чей ход), `WordsUtil` (синонимы команд).

Алгоритм `DpSolver.buildTable()`:

1. Инициализация диагонали $dp[i][i] = c_i$, $choice[i][i] = LEFT$.
2. Для длин подмассивов $len = 2 \dots N$:
   - Для каждого i, $j = i + len - 1$:
   - Вычислить $leftGain = c_i + \min(dp[i+2][j], dp[i+1][j-1])$ (с учётом границ).
   - Вычислить $rightGain = c_j + \min(dp[i+1][j-1], dp[i][j-2])$.
   - Если $leftGain \geqslant rightGain$, то $dp[i][j] = leftGain$, $choice[i][j] = LEFT$, иначе $RIGHT$.

В методе `CoinRowGame.computerMove()` вызывается `solver.optimalMove(left, right)`, который возвращает `choice[left][right]`.

Код программы

Ниже приведены листинги основных файлов проекта.

Листинг 1: AnsiColors.java

```java
package a.slelin.work.algorithms.dp.laba4;

import lombok.AccessLevel;
import lombok.NoArgsConstructor;

/**
 * Утилитный класс с ANSI-кодами для подсветки текста в консоли.
 */
@NoArgsConstructor(access = AccessLevel.PRIVATE)
public final class AnsiColors {

    public static final String RESET = "\u001B[0m";

    public static final String RED = "\u001B[31m";

    public static final String GREEN = "\u001B[32m";

    public static final String YELLOW = "\u001B[33m";

    public static final String BLUE = "\u001B[34m";

    public static final String CYAN = "\u001B[36m";

    public static final String BOLD = "\u001B[1m";
}
```

Листинг 2: Choice.java

```java
package a.slelin.work.algorithms.dp.laba4;

public enum Choice {
    LEFT,
    RIGHT
}
```

Листинг 3: Turn.java

```java
package a.slelin.work.algorithms.dp.laba4;

public enum Turn {
    PLAYER,
    COMPUTER
}
```

Листинг 4: WordsUtil.java

```java
package a.slelin.work.algorithms.dp.laba4;

import lombok.AccessLevel;
import lombok.NoArgsConstructor;

import java.util.List;

/**
 * Утилитный класс с разной возможностью написать слово.
 */
@NoArgsConstructor(access = AccessLevel.PRIVATE)
public final class WordsUtil {

    public final static List<String> leftSet;

    public final static List<String> rightSet;

    public final static List<String> exitSet;

    public final static List<String> yesSet;

    public final static List<String> noSet;

    static {
        leftSet = List.of("l", "left", "л", "левая");

        rightSet = List.of("r", "right", "п", "правая");

        exitSet = List.of(
                "\\exit", "\\quite", "\\stop", "\\end", "\\close",
                "\\выйти", "\\выход", "\\завершить", "\\стоп", "\\конец"
        );

        yesSet = List.of("yes", "y", "да", "д");

        noSet = List.of("no", "n", "нет", "н");
    }

    public static boolean isIntoSet(String key) {
        return isIntoSet(key, "exit");
    }

    public static boolean isIntoSet(String key, String setName)
```

```
        {
        List<String> set = determineSet(setName);

        for (String str : set) {
            if (str.equalsIgnoreCase(key)) {
                return true;
            }
        }

        return false;
    }

    public static List<String> determineSet(String name) {
        return switch (name) {
            case "left" -> leftSet;
            case "right" -> rightSet;
            case "exit" -> exitSet;
            case "yes" -> yesSet;
            case "no" -> noSet;
            default -> throw new IllegalArgumentException("
                Illegal set name: " + name);
        };
    }
}
```

Листинг 5: CoinRowGame.java (основная логика и DP)

```
package a.slelin.work.algorithms.dp.laba4;

import lombok.Getter;

import java.util.Arrays;

/**
 * Класс, представляющий игру "Монетный ряд".
 */
public class CoinRowGame {

    private final int[] coins;          // номиналы монет
    private int left;                   // текущий левый индекс
    private int right;                  // текущий правый индекс

    @Getter
    private int playerScore;            // счёт игрока

    @Getter
    private int computerScore;         // счёт компьютера
```

```java

    @Getter
    private Turn turn;        // чей сейчас ход
    private final DpSolver solver;     // алгоритм для решения динамической задачи

    public CoinRowGame(int[] coins) {
        if (coins == null || coins.length == 0) {
            throw new IllegalArgumentException("Ряд монет не может быть пустым.");
        }

        this.coins = Arrays.copyOf(coins, coins.length);
        this.left = 0;
        this.right = coins.length - 1;
        this.playerScore = 0;
        this.computerScore = 0;
        this.turn = Turn.PLAYER;        // первым ходит игрок
        this.solver = new DpSolver(this.coins);
    }

    public boolean isGameOver() {
        return left > right;
    }

    public int[] getCurrentCoins() {
        if (left > right) return new int[0];
        return Arrays.copyOfRange(coins, left, right + 1);
    }

    /**
     * Ход игрока: взять монету с левого или правого конца.
     */
    public void playerMove(Choice choice) {
        if (turn == Turn.COMPUTER) {
            throw new IllegalStateException("Сейчас не ваш ход.");
        }

        if (isGameOver()) {
            throw new IllegalStateException("Игра уже завершена.");
        }

        int taken = choice == Choice.LEFT ? coins[left++] : coins[right--];
        playerScore += taken;
```

```java
        turn = Turn.COMPUTER;

        System.out.println("\uD83D\uDE80 Вы выбрали " + AnsiColors.BLUE + AnsiColors.BOLD
                + (choice == Choice.LEFT ? "левую" : "правую") + AnsiColors.RESET
                + " монету достоинством : "
                + AnsiColors.GREEN + AnsiColors.BOLD + taken + AnsiColors.RESET + ".");
        System.out.println();
    }

    /**
     * Ход компьютера: выбирает оптимальный конец с помощью DP.
     */
    public void computerMove() {
        if (turn == Turn.PLAYER) {
            throw new IllegalStateException("Сейчас ход игрока, компьютер не может ходить.");
        }

        if (isGameOver()) {
            throw new IllegalStateException("Игра уже завершена.");
        }

        // Получаем рекомендацию от DP-решателя
        Choice choice = solver.optimalMove(left, right);
        int taken = choice == Choice.LEFT ? coins[left++] : coins[right--];
        computerScore += taken;
        turn = Turn.PLAYER;

        System.out.println("\uD83D\uDCCA Компьютер выбирает " + AnsiColors.BLUE + AnsiColors.BOLD
                + (choice == Choice.LEFT ? "левую" : "правую") + AnsiColors.RESET
                + " монету достоинством : "
                + AnsiColors.GREEN + AnsiColors.BOLD + taken + AnsiColors.RESET + ".");
        System.out.println();
    }

    /**
     * Вложенный класс, реализующий динамическое программирование
     * для задачи оптимальной игры в монетный ряд.
```

```java
     */
    private static class DpSolver {
        private final int[] coins;
        private final int n;
        private final int[][] dp;    // dp[i][j]     максимальный выигрыш первого игрока на подмассиве i..j
        private final Choice[][] choice;

        public DpSolver(int[] coins) {
            this.coins = coins;
            this.n = coins.length;
            this.dp = new int[n][n];
            this.choice = new Choice[n][n];
            buildTable();
        }

        /**
         * Заполняет таблицы DP и choice.
         * dp[i][j] = max( coins[i] + min(dp[i+2][j], dp[i+1][j-1]),
         * coins[j] + min(dp[i+1][j-1], dp[i][j-2]) )
         * (Учитываем, что противник тоже играет оптимально.)
         */
        private void buildTable() {
            // Базовый случай: один элемент
            for (int i = 0; i < n; i++) {
                dp[i][i] = coins[i];
                choice[i][i] = Choice.LEFT; // неважно, берём левую (единственную)
            }

            // Подмассивы длины 2 и более
            for (int len = 2; len <= n; len++) {
                for (int i = 0; i <= n - len; i++) {
                    int j = i + len - 1;

                    // Вариант 1: берём левую монету coins[i]
                    int leftGain = coins[i];
                    if (i + 1 <= j) {
                        // Противник после нашего хода будет выбирать лучшее для себя,
                        // что оставит нам минимум из двух возможных продолжений.
                        int opponentLeft = (i + 2 <= j) ? dp[i + 2][j] : 0;
                        int opponentRight = (i + 1 <= j - 1) ? dp[i + 1][j - 1] : 0;
```

```java
                        leftGain += Math.min(opponentLeft, opponentRight);
                    }

                    // Вариант 2: берём правую монету coins[j]
                    int rightGain = coins[j];
                    if (i <= j - 1) {
                        int opponentLeft = (i + 1 <= j - 1) ? dp[i + 1][j - 1] : 0;
                        int opponentRight = (i <= j - 2) ? dp[i][j - 2] : 0;
                        rightGain += Math.min(opponentLeft, opponentRight);
                    }

                    if (leftGain >= rightGain) {
                        dp[i][j] = leftGain;
                        choice[i][j] = Choice.LEFT;   // выгодне взять левую
                    } else {
                        dp[i][j] = rightGain;
                        choice[i][j] = Choice.RIGHT;  // выгодне взять правую
                    }
                }
            }
        }

        /**
         * Возвращает оптимальный ход для текущей позиции (left, right).
         */
        public Choice optimalMove(int left, int right) {
            if (left > right) {
                throw new IllegalArgumentException("Некорректные границы.");
            }

            return choice[left][right];
        }
    }
}
```

Листинг 6: GameController.java (интерфейс)

```java
package a.slelin.work.algorithms.dp.laba4;

```

```java
import java.util.Random;
import java.util.Scanner;

/**
 * Контроллер игры: отображение состояния, обработка ввода, упра
     вление ходами.
 */
public class GameController {

    private final Scanner scanner;

    private final Random random;

    private CoinRowGame game;

    private final int delimiterLength = 100;

    public GameController() {
        this.scanner = new Scanner(System.in);
        this.random = new Random();
        this.game = null;
    }

    public void start() {
        start(random.nextInt(5, 15));
    }

    public void start(int length) {
        printWelcome();

        int[] initialCoins;

        if (length < 5 || length > 15) {
            throw new IllegalArgumentException("Длина ряда должн
                а быть от 5 до 15.");
        } else {
            initialCoins = new int[length];
            for (int i = 0; i < length; i++) {
                initialCoins[i] = random.nextInt(100);
            }
        }

        game = new CoinRowGame(initialCoins);

        while (!game.isGameOver()) {
            printState();

```

```
            if (game.getTurn() == Turn.PLAYER) {
                handlePlayerMove();
            } else {
                handleComputerMove();
            }
        }

        printGameResult();
        askReplay();
    }

    private void printWelcome() {
        String welcome = " МОНЕТНЫЙ РЯД ";
        delimiterLn(delimiterLength + welcome.length());
        delimiter(delimiterLength / 2);
        System.out.print(AnsiColors.CYAN + AnsiColors.BOLD + welcome + AnsiColors.RESET);
        delimiterLn(delimiterLength / 2);
        delimiterLn(delimiterLength + welcome.length());
        System.out.println();

        delimiterLn(delimiterLength + welcome.length());
        System.out.println("Вы и компьютер по очереди берёте монету \uD83D\uDCB0 с одного из концов ряда.");
        System.out.println("Ваша цель     \uD83E\uDD47 набрать больше очков, чем компьютер \uD83E\uDD16.");
        System.out.println("Обдумывайте \uD83E\uDD2F каждый свой шаг и удачи \uD83C\uDFAF!!!");
        delimiterLn(delimiterLength + welcome.length());
        System.out.println();
    }

    private void printState() {
        int[] currentCoins = game.getCurrentCoins();

        System.out.println(AnsiColors.YELLOW + "Текущий ряд \uD83D\uDCB5: " + AnsiColors.RESET);
        for (int i = 0; i < currentCoins.length; i++) {
            if (i == 0) {
                System.out.print(AnsiColors.GREEN + "["
                        + AnsiColors.BOLD + currentCoins[i] + AnsiColors.RESET
                        + AnsiColors.GREEN + "]" + AnsiColors.RESET);
            } else if (i == currentCoins.length - 1) {
                System.out.print(" " + AnsiColors.GREEN + "["
                        + AnsiColors.BOLD + currentCoins[i] +
```

```
                                AnsiColors.RESET
                            + AnsiColors.GREEN + "]" + AnsiColors.
                                RESET);
            } else {
                System.out.print(" " + currentCoins[i]);
            }
        }

        System.out.println();
        System.out.println(AnsiColors.BLUE + "Счёт: \uD83D\uDE00
             Вы = "
                + AnsiColors.RESET + AnsiColors.GREEN +
                    AnsiColors.BOLD + game.getPlayerScore()
                + AnsiColors.RESET + AnsiColors.BLUE + " | \
                    uD83E\uDD16 Компьютер = "
                + AnsiColors.RESET + AnsiColors.RED + AnsiColors
                    .BOLD + game.getComputerScore()
                + AnsiColors.RESET);
        System.out.println();
    }

    private void handlePlayerMove() {
        System.out.print("Ваш ход.     Введите '"
                + AnsiColors.CYAN + AnsiColors.BOLD + "Л" +
                    AnsiColors.RESET
                + "' (левая монета) или '"
                + AnsiColors.CYAN + AnsiColors.BOLD + "П" +
                    AnsiColors.RESET
                + "' (правая монета): ");
        String input = scanner.nextLine().trim();

        if (WordsUtil.isIntoSet(input, "exit")) {
            System.exit(0);
        }

        Choice choice;
        if (WordsUtil.isIntoSet(input, "left")) {
            choice = Choice.LEFT;
        } else if (WordsUtil.isIntoSet(input, "right")) {
            choice = Choice.RIGHT;
        } else {
            System.out.println(AnsiColors.RED
                    + "Ошибка: введите Л или П!"
                    + AnsiColors.RESET);
            System.out.println();
            return;
        }
```

```
        delimiterLn(delimiterLength);
        System.out.println();

        try {
            game.playerMove(choice);
        } catch (IllegalStateException e) {
            System.out.println(AnsiColors.RED + "Ошибка: " + e.getMessage() + AnsiColors.RESET);
        }
    }

    private void handleComputerMove() {
        System.out.println(AnsiColors.CYAN + "    Ход компьютера ..." + AnsiColors.RESET);

        try {
            game.computerMove();
        } catch (IllegalStateException e) {
            System.out.println(AnsiColors.RED + "Ошибка в ходе компьютера: " + e.getMessage() + AnsiColors.RESET);
        }

        // Симулируем размышления компьютера
        try {
            Thread.sleep(1000);
        } catch (InterruptedException ignore) {
        }
    }

    private void printGameResult() {
        System.out.println("\uD83C\uDFC1".repeat(36));
        System.out.print("\uD83C\uDFC1".repeat(14));
        System.out.print(AnsiColors.CYAN + AnsiColors.BOLD + " ИГРА ОКОНЧЕНА! " + AnsiColors.RESET);
        System.out.println("\uD83C\uDFC1".repeat(14));
        System.out.println("\uD83C\uDFC1".repeat(36));
        System.out.println();

        int player = game.getPlayerScore();
        int computer = game.getComputerScore();
        System.out.println(AnsiColors.BLUE + "Итоговый счёт: \uD83D\uDE00 Вы = " + AnsiColors.RESET
                + AnsiColors.GREEN + AnsiColors.BOLD + player + AnsiColors.RESET + AnsiColors.BLUE
                + ", \uD83E\uDD16 Компьютер = " + AnsiColors.RESET + AnsiColors.RED + AnsiColors.BOLD
```

```
                + computer + AnsiColors.RESET);
        System.out.println();

        if (player > computer) {
            System.out.println(AnsiColors.GREEN + AnsiColors.BOLD
                    + "\uD83C\uDF89 Поздравляем! \uD83D\uDE01 Вы победили! \uD83C\uDFC6" + AnsiColors.RESET);
        } else if (player < computer) {
            System.out.println(AnsiColors.RED + AnsiColors.BOLD
                    + "\uD83D\uDE28 Компьютер выиграл. \uD83E\uDD48" + AnsiColors.RESET);
        } else {
            System.out.println(AnsiColors.YELLOW + AnsiColors.BOLD
                    + "\uD83D\uDE11 Ничья! \uD83E\uDD14" + AnsiColors.RESET);
        }
    }

    private void askReplay() {
        System.out.print("\nХотите сыграть ещё раз? (да/нет): ");
        String answer = scanner.nextLine().trim();
        System.out.println();

        if (WordsUtil.isIntoSet(answer, "exit")) {
            System.exit(0);
        }

        if (WordsUtil.isIntoSet(answer, "yes")) {
            start();
        } else if (WordsUtil.isIntoSet(answer, "no")) {
            System.out.println(AnsiColors.CYAN + AnsiColors.BOLD + "\uD83E\uDD29 Спасибо за игру! Возвращайтесь! \uD83D\uDC4B" + AnsiColors.RESET);
            scanner.close();
        } else {
            System.out.println(AnsiColors.RED + "Ошибка: не разобрал " + answer + AnsiColors.RESET);
            askReplay();
        }
    }

    private void delimiter(int count) {
        System.out.print(AnsiColors.YELLOW + "*".repeat(count) +
```

```
                AnsiColors.RESET);
    }

    private void delimiterLn(int count) {
        delimiter(count);
        System.out.println();
    }
}
```

Листинг 7: Main.java

```
package a.slelin.work.algorithms.dp.laba4;

public class Main {
    static void main() {
        GameController controller = new GameController();
        controller.start();
    }
}
```

Пример работы

Рассмотрим игру с начальным рядом монет `[7, 2, 9, 4, 6]`. Первым ходит игрок.

Таблица *dp*, построенная алгоритмом, будет иметь вид (для наглядности показаны только ключевые значения):

$i \backslash j$	0	1	2	3	4
0	7	7	11	16	13
1		2	9	6	15
2			9	9	13
3				4	6
4					6

Таблица выбора *choice*:

$i \backslash j$	0	1	2	3	4
0	L	L	R	L	L
1		L	R	L	R
2			L	L	L
3				L	R
4					L

Игрок начинает. Если он выберет левую монету (7), компьютер ответит правой (6), затем игрок снова будет выбирать и т.д. В результате при оптимальной игре обоих компьютер гарантированно выигрывает со счётом 15:13 или 16:12 в зависимости от первого хода игрока.

Более подробно показано на рисунке 1.

На рисунке 2 показан скриншот консольного интерфейса игры.

Тестирование

Корректность работы алгоритма динамического программирования проверялась на нескольких фиксированных наборах монет путём сравнения с ручным просчётом оптимальной стратегии. Также тестировался пользовательский интерфейс: обработка некорректного ввода, смена хода, завершение игры.

Все тесты пройдены успешно.

Выводы

В ходе выполнения лабораторной работы была разработана консольная игра «Монетный ряд», в которой компьютер использует динамическое программирование для выбора оптимального хода. Изучен и реализован классический алгоритм решения антагонистической игры с полной информацией. Освоены

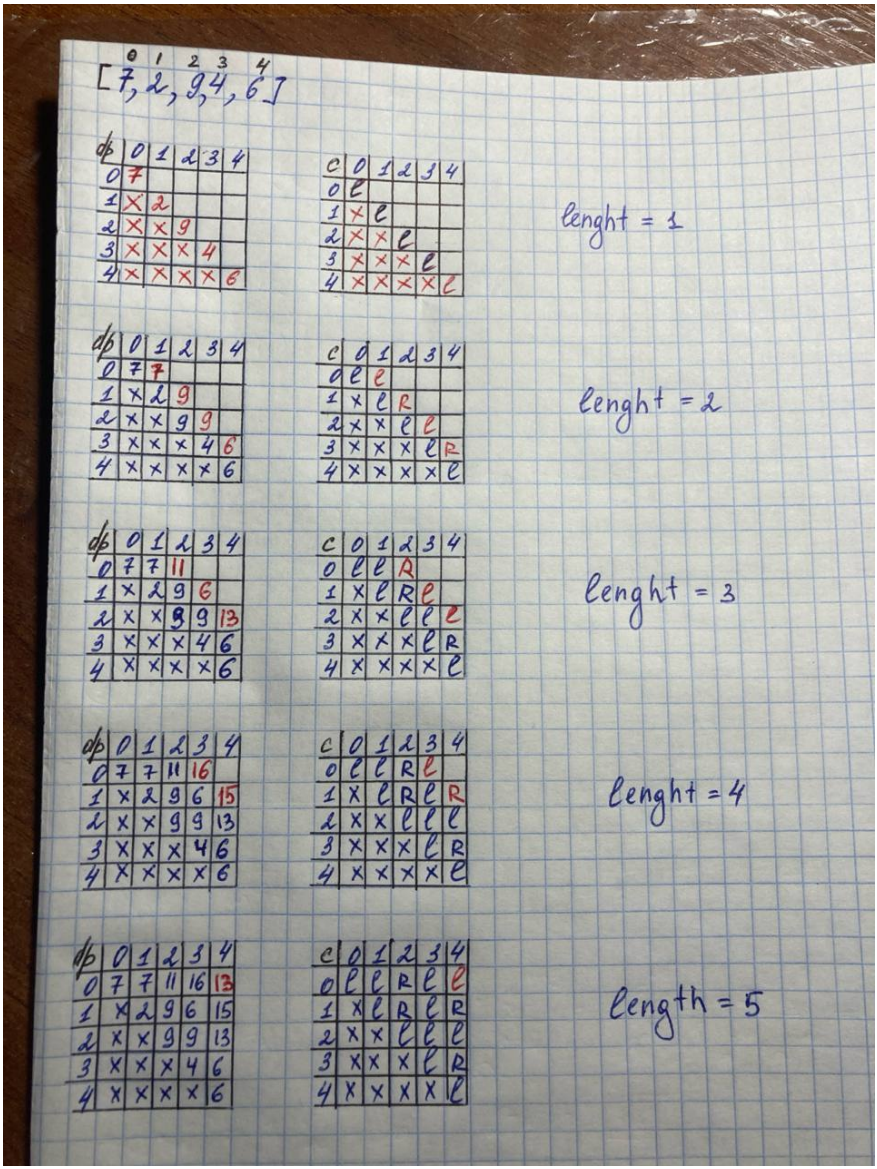

Рис. 1: Пример работы алгоритма

принципы построения таблицы оптимальных стратегий и восстановления хода. Программа имеет модульную структуру, снабжена комментариями и обрабатывает возможные ошибки ввода. Полученные навыки могут быть применены для

```
*****************************************************************************************************************
************************************************** МОНЕТНЫЙ РЯД **************************************************
*****************************************************************************************************************

*****************************************************************************************************************
Вы и компьютер по очереди берёте монету 🪙 с одного из концов ряда.
Ваша цель – 🥇 набрать больше очков, чем компьютер 🤖.
Обдумывайте 🧠 каждый свой шаг и удачи 🎯!!!
*****************************************************************************************************************

Текущий ряд 💶:
[74] 66 22 42 4 22 71 21 73 90 91 [87]
Счёт: 😀 Вы = 0 | 🤖 Компьютер = 0

Ваш ход. ⏳ Введите 'Л' (левая монета) или 'П' (правая монета): правая
*********************************************************************************************************

🚀 Вы выбрали правую монету достоинством : 87.

Текущий ряд 💶:
[74] 66 22 42 4 22 71 21 73 90 [91]
Счёт: 😀 Вы = 87 | 🤖 Компьютер = 0

⏳ Ход компьютера...
📊 Компьютер выбирает правую монету достоинством : 91.
```

Рис. 2: Пример игровой сессии

создания других игр с искусственным интеллектом, основанных на динамическом программировании.